# xG-kalkulator

Roy Kjetil Liknes 30.10.26

Kode og rapport: https://github.com/h581876/xg-kalkulator Applikasjon: https://xg-kalkulator.streamlit.app/

**Prosjektet og omfang**

Målet med prosjektet er å lage en webapplikasjon som estimerer sannsynligheten for at et skudd i fotball blir mål. Brukeren oppgir skuddets posisjon på banen, om det tas med hodet eller foten, om det kommer fra en dødball, eller om skytteren er under press og om skuddet tas på første berøring. Applikasjonen viser modellens estimerte sannsynlighet for scoring.

Prosjektet er inspirert av statistikk innenfor fotball. Expected goals (xG) er en modell som estimerer sannsynligheten for at et gitt skudd blir mål, ut fra egenskaper ved skuddet (Anzer & Bauer, 2021). Metoden brukes i prestasjonsanalyse, for eksempel til å vurdere om lag og spillere scorer mer eller mindre enn forventet. I dette prosjektet undersøker jeg hvordan en slik modell kan bygges fra bunnen med maskinlæring, og vises i et enkelt brukergrensesnitt. Prosjektet forsøker å følge livssyklusen for maskinlæring (Høgskulen på Vestlandet, n.d.).

**Hvordan løses problemet i dag:** Dataleverandører som StatsBomb og Opta publiserer xG-verdier fra egne modeller. Slike modeller bruker normalt mer informasjon enn dette prosjektet, som flere data om hendelser (Anzer & Bauer, 2021).

**Målgruppe og suksess:** Målgruppen er fotballinteresserte og personer som er interessert i fotballstatistikk. Løsningen jeg har valgt er en enkel app som bruker maskinlæring på fotballdata, ikke et ferdig verktøy for profesjonell analyse. Prosjektet regnes som vellykket hvis modellen gir bedre sannsynlighetsestimater enn en enkel baseline på tidligere usette data, og hvis estimatene er enkle å se i en fungerende webapplikasjon.

**Systemet rundt modellen:** Modellen er en del av en liten pipeline: datainnhenting med statsbombpy, datarensing og feature engineering, trening og evaluering, lagret modellfil og en Streamlit-app som laster modellen og viser sannsynligheten. Hvis featurene endres, må modellen

trenes på nytt og appens inputfelt tilpasses, ellers stemmer ikke inputen med det modellen ble trent på.

**Business objective:** Prosjektet har ingen kommersiell målsetting. Verdien ligger i læring og demonstrasjon, så ytelsen måles med modellmetrikker og at applikasjonen fungerer.

### Metrikker

Siden prosjektet skal produsere sannsynligheter og ikke bare klassifisere skudd som mål eller ikke mål, vurderes både modellens evne til å skille mellom utfallene og kvaliteten på sannsynlighetene.

- **ROC AUC** måler hvor godt modellen rangerer mål høyere enn skudd uten mål. 1 er perfekt, og rundt 0,5 tilsvarer tilfeldig rangering.
- **Log loss** måler kvaliteten på sannsynlighetene og straffer hardt når modellen er sikker og tar feil. Lavere er bedre.
- **Brier score** er gjennomsnittet av kvadrerte forskjeller mellom predikert sannsynlighet og faktisk utfall (1 for mål, 0 ellers). Det er altså mean squared error på sannsynlighetene. Lavere er bedre.
- **Accuracy, precision, recall og F1** brukes som supplerende metrikker når sannsynlighetene gjøres om til ja eller nei med en terskel (her 0,5). Siden bare ca. 10 % av skuddene er mål, ville en modell som alltid svarer ikke mål fått ca. 90 % accuracy. Derfor er ROC AUC, log loss og Brier hovedmetrikkene, og terskelen påvirker ikke xG-verdien appen viser.
- **Kalibrering** undersøkes ved å sammenligne predikert og faktisk målandel i grupper av skudd.
- **Programvare:** appen skal svare umiddelbart på endringer i input.

Modellen sammenlignes med en baseline som gir alle skudd den gjennomsnittlige målandelen i treningsdataene, og StatsBombs egne xG-verdier som referanse.

**Suksesskriterier:** modellen skal gi bedre log loss, Brier score og ROC AUC enn baseline, være godt kalibrert, og ligge nær StatsBombs xG tatt i betraktning at den bruker færre features. I tillegg skal applikasjonen fungere og kunne vises frem.

## Data

### Datakilde og labels

Prosjektet bruker StatsBomb Open Data, som publiserer kamp og hendelsesdata for utvalgte konkurranser og sesonger (StatsBomb, n.d.). Dataene hentes med Python-biblioteket statsbombpy. Programmet går gjennom alle tilgjengelige konkurranser og sesonger, henter hendelsene i hver kamp og beholder hendelser registrert som skudd.

Hver observasjon er ett skudd. Labelen er om skuddet ble mål, utledet fra feltet shot_outcome: Goal gir 1, alle andre utfall gir 0. Labelen er registrert i selve hendelsesdataene og krever ingen manuell merking, så den er konsistent for alle skudd. Dermed blir problemet supervised learning, der modellen lærer sammenhenger mellom skuddets egenskaper og det faktiske utfallet.

### Datamengde

Etter rensing består datasettet av **99 717 skudd fra 3 960 kamper**. Målandelen er **10,26 %**. Dataene er delt tilfeldig 80/20, som gir 79 773 skudd i treningssettet og 19 944 i testsettet.

### Rensing

Databehandlingen fjerner straffespark og skudd uten koordinater. Etter dette mangler ingen av featurene verdier.

### Features

Koordinatene bruker StatsBombs banesystem på 120 * 80 enheter, med målet i x = 120 og midt for mål i y = 40. Avstand er målt i disse koordinatenhetene og ikke i meter.

| Feature | Beskrivelse |
|---|---|
| dist | Avstand fra skuddposisjonen til midten av målet |
| angle | Vinkelen mellom de to stolpene sett fra skuddposisjonen, regnet ut som $\arctan2(8 \cdot dx, dx^2 + dy^2 - 16)$, der dx er avstanden til mållinjen og dy er avviket fra midten |
| is_header | 1 hvis skuddet er et hodestøt |

| Feature | Beskrivelse |
|---|---|
| set_piece | 1 hvis situasjonen kommer fra frispark eller hjørnespark |
| under_pressure | 1 hvis skytteren er under press |
| first_time | 1 hvis skuddet tas på første berøring |

**Modellering**

**Baseline og referanse**

Baseline er en modell som gir alle skudd samme sannsynlighet, lik målandelen i treningsdataene (10,3 %). StatsBombs egen xG (ikke brukt som feature) brukes som referanse fra en modell med mer informasjon.

**Modeller og valg**

Følgende modeller ble utforsket:

1. Logistisk regresjon
2. Beslutningstre med dybde 5
3. Random forest
4. Gradient boosting
5. Random forest justert med GridSearchCV

Modellene ble sammenlignet med 5-fold kryssvalidering på treningsdataene (ROC AUC), og modellen ble valgt før testsettet ble brukt:

| Modell | CV ROC AUC |
|---|---|
| Logistisk regresjon | 0,7658 |
| Beslutningstre | 0,7608 |
| Random forest | 0,6976 |
| Gradient boosting | 0,7736 |

| Modell | CV ROC AUC |
|---|---|
| Random forest (justert) | 0,7726 |

Gradient boosting hadde høyest verdi og ble valgt og lagret.

**Resultater på testsettet**

Terskel 0,5 er brukt for accuracy, precision, recall og F1.

| Modell | Log loss | ROC AUC | Brier | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|---|---|
| Baseline (gjennomsnitt) | 0,3307 | 0,5000 | 0,0921 | 0,8974 | 0,0000 | 0,0000 | 0,0000 |
| Logistisk regresjon | 0,2869 | 0,7613 | 0,0817 | 0,8993 | 0,5627 | 0,0811 | 0,1418 |
| Beslutningstre (dybde 5) | 0,2898 | 0,7616 | 0,0814 | 0,8993 | 0,5459 | 0,1105 | 0,1837 |
| Random forest | 0,6771 | 0,6915 | 0,0987 | 0,8765 | 0,2954 | 0,1471 | 0,1964 |
| **Gradient boosting** | **0,2825** | **0,7714** | **0,0809** | 0,8998 | 0,5786 | 0,0846 | 0,1475 |
| Random forest (justert) | 0,2832 | 0,7699 | 0,0811 | 0,9000 | 0,6220 | 0,0635 | 0,1153 |
| StatsBomb xG (referanse) | 0,2680 | 0,8015 | 0,0760 | 0,9035 | 0,6502 | 0,1290 | 0,2153 |

**Tolkning.** Den valgte modellen (gradient boosting) er klart bedre enn baseline. StatsBombs xG er bedre noe som gir mening siden den trolig bruker mer informasjon. De tre beste modellene (gradient boosting, finjustert random forest og logistisk regresjon) ligger nær hverandre i ROC AUC, så enkel geometri kan forklare mesteparten. Forskjellen mellom gradient boosting og justert random forest er så liten at den kan skyldes tilfeldig variasjon.

**Overfitting:** Random forest uten begrensninger er dårligere enn baseline på log loss (0,6771 mot 0,3307). Dette kan forklares med at trærne blir så dype at de gir ekstreme sannsynligheter. Etter

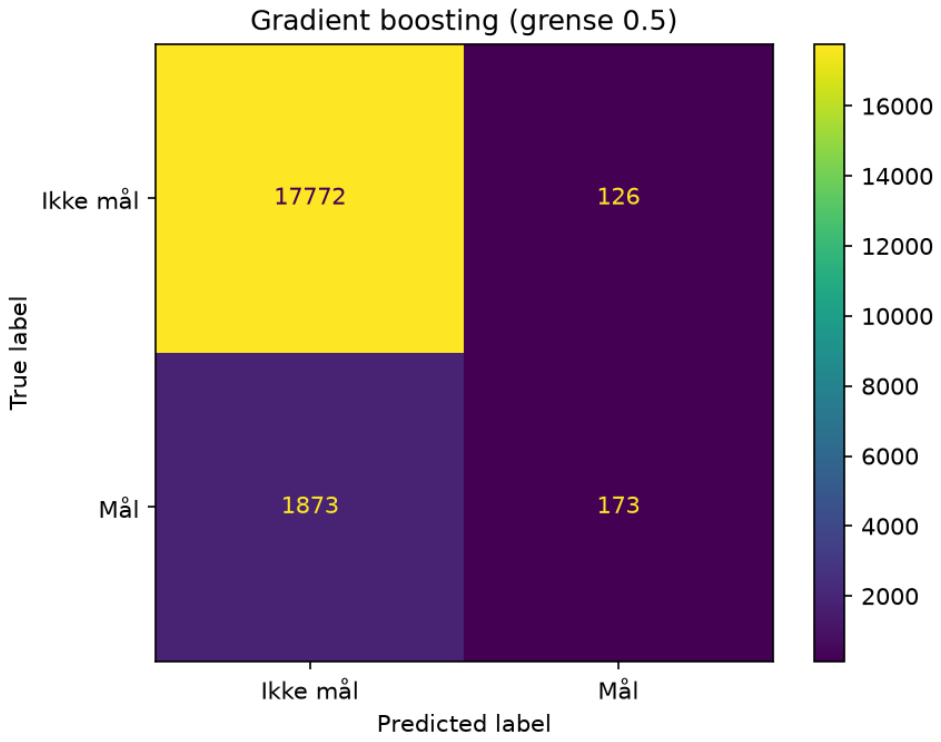
justering av max_depth og min_samples_leaf faller log loss og ROC AUC stiger. Dette kan være et eksempel på overfitting og hvordan hyperparametere fungerer på modellen.

**Terskel og confusion matrix**

Med terskel 0,5 klassifiserer den valgte modellen svært få skudd som mål: av 2 046 faktiske mål i testsettet fanges 173, og 126 skudd uten mål feilklassifiseres som mål. Når terskelen senkes, øker recall på bekostning av precision:

| Terskel | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| 0,10 | 0,690 | 0,209 | 0,723 | 0,324 |
| 0,15 | 0,793 | 0,263 | 0,561 | 0,358 |
| 0,20 | 0,848 | 0,318 | 0,421 | 0,362 |
| 0,30 | 0,890 | 0,433 | 0,220 | 0,292 |
| 0,50 | 0,900 | 0,579 | 0,085 | 0,148 |

F1 er best ved terskler rundt 0,15 til 0,20. Terskelen påvirker bare ja og nei klassifiseringen, ikke xG-verdien appen viser.




**Kalibrering og feilanalyse**

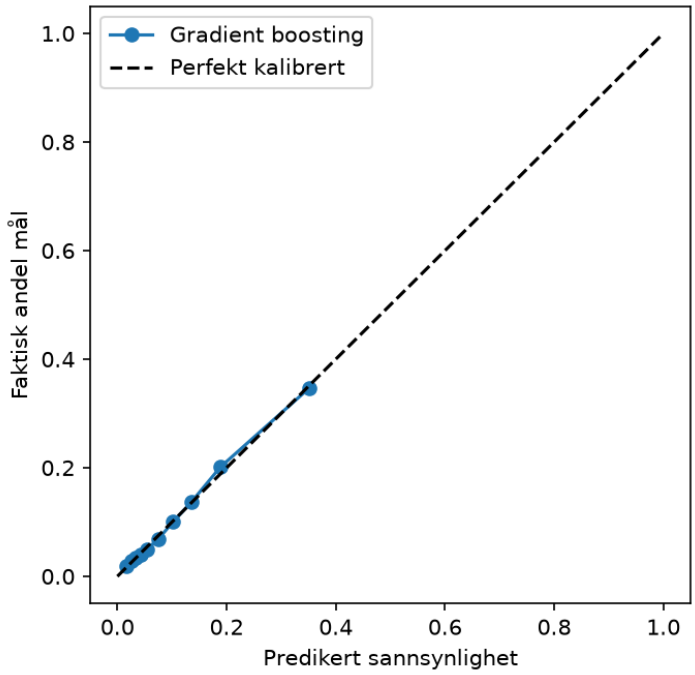
For å undersøke feilprediksjoner ble målandel sammenlignet med faktisk mål i grupper av testskudd.

| Avstand | Antall skudd | målandel | Predikert |
|---|---|---|---|
| 0 til 6 | 634 | 0,427 | 0,450 |
| 6 til 12 | 4 347 | 0,173 | 0,175 |
| 12 til 18 | 4 679 | 0,126 | 0,122 |
| 18 til 25 | 5 077 | 0,057 | 0,056 |
| over 25 | 5 207 | 0,028 | 0,027 |

| Gruppe | Antall skudd | målandel | Predikert |
|---|---|---|---|
| Fot | 16 752 | 0,102 | 0,100 |

| Gruppe | Antall skudd | målandel | Predikert |
|---|---|---|---|
| Hode | 3 192 | 0,108 | 0,115 |
| Åpent spill | 12 804 | 0,109 | 0,107 |
| Dødball | 7 140 | 0,091 | 0,094 |

Den største forskjellen er for skudd nært mål (0,450 mot 0,427), en gruppe med få skudd, og modellen ser ut til overpredikere noe for hodeskudd. Kalibreringskurven:




**Feature importance**

Feature importance fra den finjusterte random forest viser at vinkel og avstand dominerer, etterfulgt av hodestøt, dødball, første berøring og under press. Det gir mening at posisjon er den viktigste faktoren for om et skudd blir mål i fotball. Press og første berøring betyr lite i denne modellen, og de kan være dekket av avstand og vinkel. Dette kan brukes til å prioritere hvilke features som er verdt å utforske videre, for eksempel mer informasjon om forsvarere og keeper.

**Begrensninger**

- Bare seks enkle features, uten keeper- og forsvarerposisjoner.

- Data fra utvalgte konkurranser og sesonger. Det er ikke undersøkt om modellen fungerer likt for ulike konkurranser, nivåer eller kjønn.

**Deployment**

Modellen trenes offline, scriptene load_data.py og train.py henter dataene, lager features, trener og evaluerer modellene og lagrer den valgte modellen som models/model.joblib.
Webapplikasjonen er laget med Streamlit (Streamlit, 2026). Den laster modellfilen en gang og kjører en prediksjon hver gang brukeren endrer input. Brukeren velger skuddets posisjon med to slidere (x og y), kroppsdel (fot eller hode), situasjon (åpent spill eller dødball) og krysser av for press og første berøring. Appen regner ut avstand og vinkel på samme måte som under trening og dermed viser sannsynligheten for mål og tegner skuddets plassering på en bane.

**Referanser:**


- Anthropic. (2026). *Claude* (Claude Sonnet 5.5) https://claude.ai
  Claude er brukt som AI-assistent til veiledning om oppsett, metode og noe kodeforslag.
- Anzer, G. & Bauer, P. (2021). A goal scoring probability model for shots based on synchronized positional and event data in football. *Frontiers in Sports and Active Living, 3*, Article 624475. https://doi.org/10.3389/fspor.2021.624475
- DAT158, forelesningsnotater og notebooks (HVL): Lecture 9, ML project lifecycle. https://hvl-ml.github.io/DAT158/slides/3-systems/9-ml-project-lifecycle.html og https://github.com/HVL-ML/DAT158
- StatsBomb. (n.d.). *StatsBomb open data*. GitHub. https://github.com/hudl/open-data
- Streamlit (2026) (Version 1.65.0). https://streamlit.io